



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

HTML e CSS

Maura Pintor

maura.pintor@unica.it

In questa lezione

Gli ipertesti e l'HTML

I tag HTML

Elementi dei documenti HTML

Gli stili e il rendering

Fogli di stile CSS



Visualizzazione degli ipertesti

Inizialmente il Web è stato sviluppato per condividere documenti ipertestuali

I protocolli inizialmente permettevano solo la condivisione di documenti *statici*, ovvero pagine preparate precedentemente

Queste pagine sono visualizzabili e navigabili utilizzando opportune applicazioni che sono chiamate **browser web** (Chrome, Firefox, Safari, ...)

Successivamente furono definiti strumenti capaci di generare pagine in modo *dinamico*, per esempio con dati personalizzati per ogni utente



Linguaggi di markup

I linguaggi di markup sono composti da:

- un insieme di istruzioni chiamate **tag** che rappresentano le caratteristiche del formato per il documento
- una grammatica
- una semantica

I tag vengono usati direttamente all'interno del testo in cui vengono applicati e modificano come il testo viene *visualizzato* dal browser

<p>Testo di contenuto </p>

I tag includono dei caratteri speciali che li *delimitano* e permettono al browser di distinguere il testo del contenuto dal testo che contiene i tag

Linguaggi di markup

Ci sono due tipi di linguaggi di markup:

- i linguaggi di **tipo procedurale** indicano le procedure di trattamento del testo aggiungendo le istruzioni che devono essere eseguite per visualizzare la porzione di testo referenziata (troff, TEX, ...)
- i linguaggi di **tipo descrittivo** lasciano la scelta del tipo di rappresentazione da applicare al testo al software che di volta in volta lo riprodurrà (SGML, HTML, XML, ...)

I linguaggi del secondo tipo si concentrano sui problemi strutturali di leggibilità e prescindono in fase di lettura dal software con cui sono stati generati

Permettono di garantire *separazione tra struttura* (o contenuto) *e visualizzazione* (o presentazione)

Noi vedremo prevalentemente i linguaggi **descrittivi** in questo corso

Hypertext Markup Language (HTML)

L'HTML è un **linguaggio di markup di tipo descrittivo** usato per descrivere un ipertesto

- usato solo per rappresentazione del layout, non è un linguaggio di programmazione!

Il browser comprende questo formato ed elabora il testo e lo visualizza (rendering)

Tramite HTML si possono realizzare documenti con elementi interattivi (link, contenuti multimediali, etc.), associando anche il significato grafico ai componenti, ovvero istruzioni su come *visualizzare*

HTML5

HTML5 è uno standard di tipo “living standard”, ovvero che viene aggiornato periodicamente

Si possono aggiungere nuove funzionalità ma le vecchie devono sempre essere mantenute e compatibili

La prima pubblicazione risale al 2012 e venne fatta dal Web Hypertext Application Technology Working Group (WHATWG)

La storia di HTML5 (e predecessori / competitor) è molto complicata e cosparsa di anti-standard, ovvero sviluppi indipendenti che hanno portato a diverse scissioni

Non guarderemo le varie versioni nel dettaglio (si trova tutto su Wikipedia per i curiosi), ma ci concentreremo sulle funzionalità necessarie al nostro corso

I tag HTML

I tag HTML sono delimitati da due caratteri: "<" all'inizio e ">" che chiude il tag (parentesi angolari)

Sono normalmente accoppiati, con un tag di inizio (*start tag*) e un tag di fine (*end tag*)

Il testo tra start ed end tag è il *contenuto* dell'elemento

<p>	Testo di contenuto	</p>
start tag	contenuto	end tag

- Ci sono delle eccezioni, come degli elementi senza chiusura, per esempio
 (new line)
- I tag si possono annidare

<p> 	Testo di contenuto	 </p>
---------	--------------------	-----------

Entità HTML

HTML definisce dei caratteri "riservati" che vengono usati per la sintassi, e dunque rimpiazza quei caratteri (qualora dovessero essere usati nel testo) con dei codici predefiniti

Per esempio, se dovessi usare il "<" per indicare un "minore di", non potrei direttamente usarlo qui perché sarebbe inteso come inizio di un tag

Per questo in HTML ci sono entity names e entity numbers (alternativi)

- *&entity_name;* per esempio *<*
- *&#entity_number;* per esempio *<*

Result	Description	Name	Number
	non-breaking space	 	
<	less than	<	<
>	greater than	>	>
&	ampersand	&	&
"	double quotation mark	"	"
'	single quotation mark	'	'
¢	cent	¢	¢
£	pound	£	£
¥	yen	¥	¥
€	euro	€	€
©	copyright	©	©
®	trademark	®	®

Attributi HTML

Gli attributi dettagliano gli elementi HTML con coppie chiave-valore
I valori sono racchiusi da apici singoli o doppi

```
<a href="https://www.google.com">Visit Google</a>
```

Struttura base di un documento HTML

La struttura base di un documento HTML è composta da 5 elementi:

```
<!DOCTYPE html>
<html>

  <head>
    <title> Title here </title>
  </head>

  <body>
    Web page content goes here.
  </body>

</html>
```

- <!DOCTYPE>
- <html>
- <head>
- <title>
- <body>

Li vediamo uno a uno nelle prossime slide

Document Type Definition (DTD)

Il DTD serve a comunicare al browser il tipo di documento così da poter identificare le regole di interpretazione e visualizzazione

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"  
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

Per esempio, specifica la versione di HTML e altri aspetti relativi alle regole di accesso (public), e altre specifiche di standard

L'elemento <html>

Questo elemento specifica al browser che la pagina sarà formattata in HTML, e opzionalmente in che lingua saranno i contenuti

Questo serve, per esempio, a spezzare le sillabe delle parole

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
```

The <html> element tells the browser that the page will be formatted in HTML and, optionally, which world language the page content is in.

L'header HTML

L'header (delimitato dal tag <head>) contiene i metadati per il documento, che **non sono visualizzati** all'interno del documento (ma alcuni impostano visualizzazioni particolari nel browser)

Il tag <title> è un esempio di questi metadati, e specifica il titolo della pagina che sarà mostrato nella testata della finestra o tab del browser

Il tag <meta> contiene metadati (dati sui dati) e informazioni utili ad applicazioni esterne, come per esempio i motori di ricerca

```
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<title>Page Title Goes Here</title>
</head>
<body>
<!-- Page content goes here -->
</body>
```

Elementi aggiuntivi e collegamenti a file esterni

Gli elementi esterni vengono inclusi nell'HTML dentro l'header, tramite i campi:

- `<link>` collegamenti verso file esterni
- `<script>` codice eseguibile utilizzato dal documento (solitamente JavaScript)
- `<style>` informazioni di stile (CSS locali)

Vedremo alcuni di questi elementi più avanti nel corso.

Il contenuto dell'HTML

Il contenuto è specificato dentro i tag <body>

Questa è la parte che viene visualizzata dal browser, e costituisce l'elemento radice in cui possono essere messi altri elementi anche in modo annidato:

- intestazioni (titoli)
- strutture di testo come paragrafi
- elenchi e liste
- tabelle
- moduli elettronici (form)
- collegamenti ipertestuali
- immagini e contenuti multimediali
- contenuti interattivi

Non vedremo tutti questi in dettaglio, ma una guida abbastanza esaustiva si trova online

<https://www.w3schools.com/html/>



I tag <div>

Il tag <div> è il divisore del documento, ovvero definisce dei blocchi raggruppati di elementi

Li vedremo anche come elementi per dare una struttura in righe e colonne alla pagina (come alternativa migliore delle tabelle in HTML)

Sono molto usati in combinazione agli stili e agli script per manipolarne il contenuto

Se non viene associato a uno stile (per esempio un background) è invisibile

stile {

```
<html>
<head>
<style>
.myDiv {
  border: 5px outset red;
  background-color: lightblue;
  text-align: center;
}
</style>
</head>
<body>
```

div con stile {

```
<div class="myDiv">
  <h2>This is a heading in a div element</h2>
  <p>This is some text in a div element.</p>
</div>
</body>
</html>
```

Il tag `<span>`

Lo `<span>` è un contenitore generico che può essere annidato

Anche questo viene usato in combinazione agli stili per dare aspetti particolari ai pezzi di testo

Anche questo è invisibile a meno che non sia associato a un foglio di stile

La differenza col `div` è che questo è inline, non mette una nuova linea tra un elemento e l'altro

Collegamenti (link) ipertestuali

Il tag `<a>` (anchor) viene usato per collegare un documento HTML a un altro o a un altro punto dello stesso documento

Il link è composto da due estremi detti "anchors"

Source anchor (ancora sorgente) → destination anchor (ancora destinazione)

Il source anchor è contenuto nella pagina di partenza, mentre il destination anchor è un'altra risorsa web

`<a href="www.google.com">Visita Google</a>`

destination anchor parte visualizzata

URL relativi e assoluti

Gli URL si distinguono in assoluti e relativi

- Gli URL **assoluti** contengono direttamente il link completo per la nuova risorsa
 - es. https://it.wikipedia.org/wiki/World_Wide_Web
- Gli URL **relativi** sono risolti utilizzando come base quella del documento corrente
 - es. se sono già su Wikipedia, le altre risorse possono essere definite in modo relativo alla pagina corrente

```
<figure class= mw-default-size type= mw:File/Thumb >...</figure> [Event]
▼ <p> = $0
  "Il "
  <b>World Wide Web</b>
  " (termine in "
  <a href="/wiki/Lingua_inglese" title="Lingua inglese">lingua inglese</a>
  " traducibile in "
  <a href="/wiki/Lingua_italiana" title="Lingua italiana">italiano</a>
  " come "rete di ampiezza mondiale" o "rete mondiale"), abbreviato "
  <h>web</h>
```

Metodi di “studio” dell’HTML

Sconsigliato imparare tutti i tipi di tag e tutte le possibili combinazioni

Meglio sapere i tipi principali e sapere dove guardare per capire come usarli

- **learn by doing**: faremo degli esercizi dove proviamo a capire come usare i tag HTML più comuni direttamente visualizzandoli sul nostro browser



Esempio di HTML

Iniziamo a costruire un HTML e indicare degli elementi

- titoli e paragrafi
- una lista puntata
- una tabella
- un menu a tendina
- dei link

Header e inizio

```
<!DOCTYPE html>
```

```
<html lang="it">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Il mio profilo</title>
```

```
</head>
```

Contenuti dell'HTML: header, paragrafi, liste puntate

```
<body>
<div>
<h1>Esempio di HTML</h1>
<p>Pagina di test</p>
<p>Esempio semplice di HTML per il corso di Web Programming.</p>
<h3>Lista puntata:</h3>
<ul>
<li>Elemento 1</li>
<li>Elemento 2</li>
<li>Elemento 3</li>
</ul>
```



Contenuti dell'HTML: tabelle

```
<h3>Tabella:</h3>
<table>
<tr>
<th>Anno</th>
<th>Evento</th>
<th>Descrizione</th>
</tr>
<tr>
<td>1 Febbraio 2025</td>
<td>Corso di Web Programming</td>
<td>Lezione 1</td>
</tr>
<tr>
<td>2 Febbraio 2025</td>
<td>Corso di Web Programming</td>
<td>Lezione 2</td>
</tr>
</table>
```

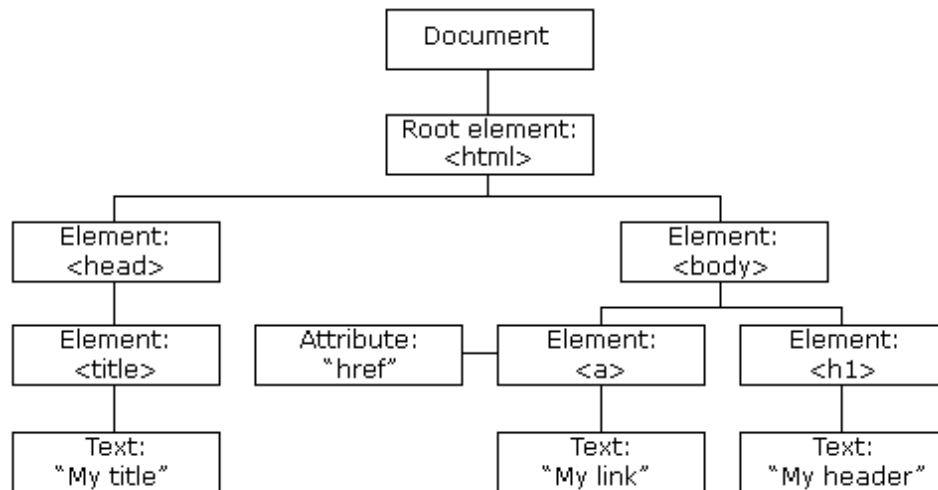
Contenuti dell'HTML: menu a tendina e link

```
<h3>Seleziona un Progetto:</h3>
<select>
  <option value="">Scegli un progetto</option>
  <option value="1">Opzione 1</option>
  <option value="1">Opzione 2</option>
  <option value="1">Opzione 3</option>
</select>
<div>
  <a href="https://github.com/unica-web" target="_blank">GitHub</a>
</div>
</div>
</body>
</html>
```

Document Object Model (DOM)

La pagina HTML si può rappresentare con una struttura ad albero

```
<html>
  <head>
    <title>My title</title>
  </head>
  <body>
    <a href="...">MyLink</a>
    <h1>My header</h1>
  </body>
</html>
```



Aggiungere lo stile all'HTML

Abbiamo già visto che si possono definire degli stili da usare per ogni elemento del documento HTML

Questo approccio element-wise può essere utile se l'elemento da modificare è solo uno, tuttavia diventa scomodo se lo stesso stile deve essere applicato più volte (per esempio in una serie di pulsanti che devono avere tutti lo stesso aspetto)

Per questo motivo sono stati introdotti i **Cascading Style Sheet** (CSS, fogli di stile a cascata)

I CSS separano il contenuto dalla presentazione, definendo come i diversi **tipi di elemento** devono essere presentati all'utente

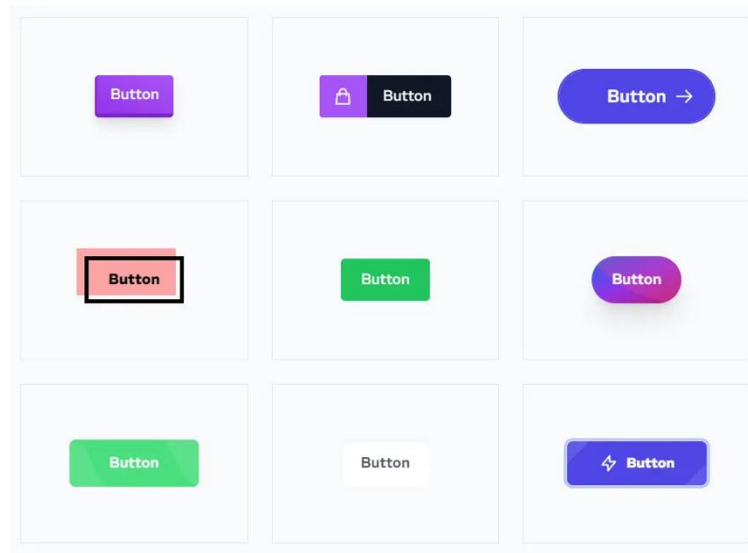
Il termine cascading è legato al fatto che possono essere inseriti più fogli di stile che vengono applicati uno dopo l'altro **in cascata**

Vantaggi dei CSS

La separazione di stile e contenuto consente di **riusare** lo stesso stile per parti diverse del documento (anche all'interno di pagine diverse) senza necessità di ridefinirle

Si possono creare stili diversi che cambiano a seconda del **contesto** (per esempio a seconda del dispositivo che visualizza, per esempio lo smartphone)

Oltretutto questo velocizza anche il caricamento della pagina, visto che i file di stile possono rimanere in **cache** nel dispositivo ed evitare di essere ricaricati tra una pagina e l'altra



Come legare il CSS all'HTML

Senza CSS, l'HTML ha una sua visualizzazione di base

Creiamo per esempio un paragrafo con il tag <p>

Per caricare lo stile usiamo il tag di collegamento <link>

Titolo pagina

Paragrafo di testo.

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Titolo pagina</h1>
    <p>Paragrafo di testo.</p>
  </body>
</html>
```

Come legare il CSS all'HTML

Poi creiamo un altro file, chiamato "styles.css", e lo mettiamo nella stessa cartella

Questo definisce gli stili per gli elementi dell'HTML

```
body {  
    background-color: powderblue;  
}  
h1 {  
    color: blue;  
}  
p {  
    color: red;  
}
```

Titolo pagina

Paragrafo di testo.

Regole CSS

Le regole CSS sono composte da espressioni con una sintassi precisa

- Selettore: specifica l'elemento o i tipi di elementi (es. tutti i <p>) a cui si vuole applicare lo stile
- Dichiarazione: composta da coppie chiave-valore
 - Proprietà: cosa si vuole cambiare
 - Valore: come si vuole cambiare

Si possono anche raggruppare, es. specificare più elementi o più valori insieme, separati da virgola

```
selettore {  
  proprietà1: valore1;  
  proprietà2: valore2;  
}
```

```
p {  
  color: red;  
  text-align: center;  
}
```


Regole CSS

Esistono anche dei modi avanzati per definire specifici elementi o gruppi di elementi, denominati **classi**

- Specifici elementi si identificano nell'HTML tramite l'attributo `id="nome_id"`

```
.nome_id {  
  color: red;  
  text-align: center;  
}
```

- Si identificano nell'HTML tramite l'attributo `class="nome_classe"`

```
.nome_classe {  
  color: red;  
  text-align: center;  
}
```

- Ci sono poi ulteriori costrutti come le pseudoclassi (es. link visitati o hover) e pseudo elementi (es. la prima linea o la prima lettera di un paragrafo)

Valori per specificare dimensioni

Per specificare i valori numerici per le dimensioni, ci sono diverse opzioni

- Unità di misura **relative**
 - em: relativo al font size dell'elemento ($2em = 2$ volte il font)
 - %: relativo all'elemento genitore
 - px: relativi al dispositivo di visualizzazione
- Unità di misura **assolute**
 - in: pollici (inches)
 - cm: centimetri
 - mm: millimetri
 - pt: points ($1pt = 0.35$ mm)
 - pc: picas ($1pc = 12$ pt)

Come specificare i colori

I colori possono essere specificati con diversi metodi

- in forma esadecimale, per esempio #0000FF
- tramite la funzione per specificare i valori RGB, per esempio `rgb(0, 0, 255)`
- usando una serie di parole chiave con colori predefiniti, per esempio "blue"

Una buona regola di design è non avere troppi colori, ma riusare sempre gli stessi in modo consistente

Ci sono siti per generare palette di colori in modo automatico



Applicazione degli stili agli elementi

Sono definiti degli stili di default per tutti gli elementi, infatti ogni elemento deve avere uno stile per essere visualizzato

L'attribuzione degli stili può essere:

- Diretta, ovvero specificata per il tipo di elemento in oggetto, oppure
- Indiretta, ereditata dall'elemento che lo contiene (solo per alcune proprietà)
- Si possono ridefinire stili che già sono definiti da altri CSS (override)

Cosa succede in caso di **conflitti**? Per esempio, un selettore che indica l'id elemento con uno stile e l'altro che invece indica il tipo di elemento con un altro stile. Quale ha precedenza?

Specificità e priorità delle regole CSS

La risoluzione viene fatta con un meccanismo che prende il nome di **specificity**

L'elemento con la più alta specificity vince e il suo stile sarà applicato all'elemento HTML

La logica si basa sulla gerarchia degli elementi e su quanto sono specifici

- Il selettore di classe è **più specifico** del selettore di paragrafo, quindi avrà priorità
- Rispetto alla classe, il **selettore di id** ha ancora più specificità
- Ancora più specifico è lo **stile inline**, che ha specificità più alta di tutti
- Infine, se esistono due regole con la stessa specificità, verrà applicata l'ultima dichiarata

```
<html>
<head>
  <style>
    .test {color: green;}
    p {color: red;}
  </style>
</head>
<body>

<p class="test">Hello!</p>

</body>
</html>
```

Aggiungere il CSS al file HTML

```
<style>
body {
display: flex;
justify-content: center;
align-items: center;
min-height: 100vh;
margin: 0;
background-color: #f0f2f5;
font-family: Arial, sans-serif;
}

.card {
background-color: white;
border-radius: 10px;
padding: 20px;
text-align: center;
width: 400px;
box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

img {
width: 150px;
height: 150px;
border-radius: 50%;
margin-bottom: 15px;
}
```



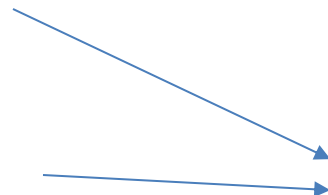
Aggiungere il CSS al file HTML

```
h1 {  
  color: #1a1a1a;  
  margin: 10px 0;  
  font-size: 24px;  
}
```

```
.titolo {  
  color: #4a90e2;  
  font-weight: bold;  
  margin: 5px 0;  
}
```

```
.descrizione {  
  color: #666;  
  font-size: 14px;  
  line-height: 1.5;  
  margin: 15px 0;  
}
```

```
.contatti {  
  display: flex;  
  justify-content: center;  
  gap: 15px;  
  margin-top: 20px;  
}
```



```
<div class="card">  
  <h1>Esempio di HTML</h1>  
  <p class="titolo">Pagina di test</p>  
  <p class="descrizione">Esempio semplice di HTML per il corso di Web Programming.</p>  
</div>
```

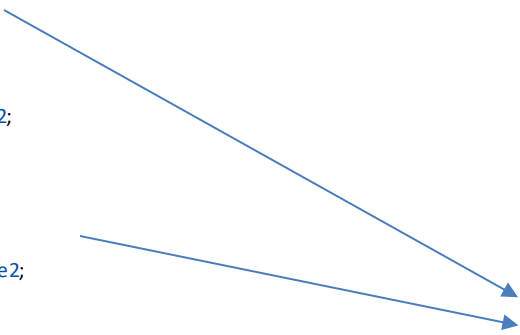
Aggiungere il CSS al file HTML

```
.contatti a {  
  color: #4a90e2;  
  text-decoration: none;  
  padding: 8px 15px;  
  border: 1px solid #4a90e2;  
  border-radius: 20px;  
  transition: all 0.3s ease;  
}
```

```
.contatti a:hover {  
  background-color: #4a90e2;  
  color: white;  
}
```

```
ul {  
  text-align: left;  
  margin: 20px 0;  
  padding-left: 30px;  
}
```

```
li {  
  margin: 10px 0;  
  color: #444;  
}
```



```
<div class="contatti">  
  <a href="https://github.com/unica-web"  
    target="_blank">GitHub</a>  
</div>
```


Aggiungere il CSS al file HTML

```
table {  
width: 100%;  
margin: 20px 0;  
border-collapse: collapse;  
}  
  
th, td {  
padding: 10px;  
border: 1px solid #ddd;  
}  
  
th {  
background-color: #f5f5f5;  
}  
  
select {  
padding: 8px;  
margin: 10px 0;  
border-radius: 5px;  
border: 1px solid #ddd;  
}  
</style>
```

